Introduction to Javascript

JavaScript is a high-level, interpreted, and dynamic programming language that adds interactivity, logic, and functionality to web pages. It allows developers to build responsive, user-friendly websites that can react to user actions, update content in real time, and control animations or multimedia. JavaScript runs as a single-threaded, synchronous language, meaning it executes one task at a time — one line after another.

Even though JavaScript runs on a single thread, it can still handle many tasks efficiently using asynchronous programming with the event loop and callback queue. This allows it to manage slow operations—like fetching data or waiting for user actions—without stopping the rest of the code from running. Brendan Eich created JavaScript in 1995, and today it follows the ECMAScript (ES) standard, which defines how the language works and keeps it consistent across all web browsers.

Features of JavaScript

1. **Lightweight and Interpreted:** No compilation required — executed line by line by the browser.
2. **Dynamic Typing:** Variable types change automatically at runtime.
3. **Object-Oriented:** Supports objects, classes, and prototypes.
4. **Platform Independent:** Runs on any device with a JS engine.
5. **Event-Driven:** Responds to user actions like clicks or keypresses.
6. **Single-Threaded but Asynchronous:** Uses event loop for concurrency.
7. **Functional & Procedural:** Multiple programming styles supported.
8. **Client & Server-Side:** Works on browsers and servers (Node.js).

JavaScript Data Types

JavaScript uses data types to define the kind of value a variable holds, like numbers or text. It's dynamically typed, so the data type is decided automatically at runtime.

JavaScript data types are categorized into two major groups:

1. Primitives
2. Non Primitives

1. Primitive data types:

Primitive types represent single values and are stored directly in memory.
They are immutable, which means their values cannot be changed once created.

Type	Description	Example
string	Represents text	"Hello"
number	Numeric values	42, 3.14
boolean	Logical true/false	true, false
undefined	Declared but unassigned variable	let a;
null	Intentional absence of value	null
symbol	Unique identifier (ES6)	Symbol('id')
bigint	Large integers (ES2020)	123n

Example :

```
// 1. String
let name = "Hello";
console.log("String:", name);
// 2. Number
let age = 42;
```

```
let pi = 3.14;
console.log("Number:", age, pi);

// 3. Boolean
let isActive = true;
let isAdmin = false;
console.log("Boolean:", isActive, isAdmin);

// 4. Undefined
let notAssigned;
console.log("Undefined:", notAssigned);

// 5. Null
let emptyValue = null;
console.log("Null:", emptyValue);

// 6. Symbol
let uniqueId = Symbol('id');
console.log("Symbol:", uniqueId);
let sym1 = Symbol('id');
let sym2 = Symbol('id');
console.log(sym1, sym2);
console.log(sym1 === sym2); // false
// 7. BigInt
let largeNumber = 1234567890123456789012345678901234567890n;
console.log("BigInt:", largeNumber);
```

2. Non-Primitive (Reference) Types :

These are objects that store multiple or complex values and are stored by reference, meaning variables point to the same memory location

Type	Description	Example
Object	Collection of key-value pairs	{name: "John", age: 22}

Array	Ordered list	[1, 2, 3]
Function	Reusable code block	function greet() {}

Example :

```
// 1. Object
let person = { name: "John", age: 22 };
console.log("Object:", person);
console.log("Person's Name:", person.name); // Accessing a property

// 2. Array
let numbers = [1, 2, 3];
console.log("Array:", numbers);
console.log("First number:", numbers[0]); // Accessing an element

// 3. Function
function greet(name) {
  return "Hello, " + name + "!";
}
console.log("Function call:", greet("Alice"));
```

JavaScript Variables:

A JavaScript variable is a container used to store data values like numbers, strings, or objects. It allows you to reuse and manipulate data throughout your program.

1. var

- Old way to declare variables.
- Function-scoped (visible throughout the function).

- Can be redeclared and reassigned.
- Hoisted and initialized as `undefined` (can be used before declaration, but not recommended).

```
console.log(a); // Hoisted but gives undefined (declared later)
var a = 10;     // Declaration and initialization

function demoVar() {
  var b = 20;          // Function-scoped
  var b = 30;          // Redeclared in the same scope
  b = 40;              // Reassigned
  console.log("Inside function:", b);
}

demoVar();

console.log("Outside function:", a); // Accessible here
// console.log(b); Error: b is not defined (function-scoped)
```

2. let

- Introduced in ES6.
- Block-scoped (works only inside { }).
- Can be reassigned but not redeclared in the same scope.
- Hoisted but not initialized (accessing before declaration causes *ReferenceError*).

```
// console.log(x); // ReferenceError: Cannot access 'x' before initialization
let x = 10;        // Declared using let (introduced in ES6)

{
  let y = 20;      // Block-scoped: works only inside this block
  y = 25;          // Can be reassigned
  console.log("Inside block:", y);
  // let y = 30;   // Error: cannot redeclare in the same block
}
```

```
// console.log(y); // Error: y is not defined (block-scoped)
console.log("Outside block:", x);
```

3. const

- Introduced in ES6.
- Block-scoped like `let`.
- Cannot be reassigned or redeclared.
- Hoisted but not initialized (using before declaration causes *ReferenceError*).
- Must be assigned a value at the time of declaration.

```
// Trying to access before declaration causes ReferenceError
// console.log(x);

const x = 10; // Must be assigned when declared
console.log("Initial value of x:", x);

// Cannot reassign
// x = 20;

// Cannot redeclare in the same scope
// const x = 30;

{
  const y = 50; // Block-scoped
  console.log("Inside block:", y);
}

// y is not accessible outside the block
// console.log(y);
```

JavaScript Operators:

Operators are special symbols used to perform operations on operands (values and variables). JavaScript has several types of operators.

Assignment Operators

Assignment operators are used to store values in variables or to perform operations while assigning.

Operator	Description	Example	Same As
=	Assigns value	x = 5	x = 5
+=	Adds and assigns	x += 3	x = x + 3
-=	Subtracts and assigns	x -= 2	x = x - 2
*=	Multiplies and assigns	x *= 4	x = x * 4
/=	Divides and assigns	x /= 2	x = x / 2
%=	Modulus and assigns	x %= 3	x = x % 3

Example :

```
let x = 5;          // = assigns value
console.log("Initial value:", x);  // 5

x += 3;             // adds and assigns
console.log("After x += 3:", x);   // 8

x -= 2;             // subtracts and assigns
console.log("After x -= 2:", x);   // 6

x *= 4;             // multiplies and assigns
```

```
console.log("After x *= 4:", x);   // 24

x /= 2;              // divides and assigns
console.log("After x /= 2:", x);   // 12

x %= 3;              // modulus and assigns
console.log("After x %= 3:", x);   // 0
```

Arithmetic Operators

Used to perform mathematical calculations.

Operator	Description	Example	Output
+	Adds two numbers	10 + 5	15
-	Subtracts numbers	10 - 5	5
*	Multiplies numbers	10 * 3	30
/	Divides numbers	10 / 2	5
%	Finds remainder	10 % 3	1
**	Raises power	2 ** 3	8

```
let a = 20;
let b = 10;
console.log("a =", a, ", b =", b);

// Addition
console.log("a + b =", a + b);     // 30
// Subtraction
console.log("a - b =", a - b);     // 10
```

```
// Multiplication
console.log("a * b =", a * b);     // 200

// Division
console.log("a / b =", a / b);     // 2

// Modulus (Remainder)
console.log("a % b =", a % b);     // 0

// Exponentiation (Power)
console.log("a ** 2 =", a ** 2);   // 400
```

Comparison Operators

Comparison operators are used to compare two values and produce a Boolean result (`true` or `false`).

Operator	Description	Example	Result
==	Equal to (compares only values)	5 == '5'	true
===	Strict equal (compares value + type)	5 === '5'	false
!=	Not equal (value only)	5 != '5'	false
!==	Strict not equal (value + type)	5 !== '5'	true
>	Greater than	7 > 4	true
<	Less than	7 < 4	false
>=	Greater or equal	10 >= 10	true
<=	Less or equal	5 <= 9	true

Example :

```
let a = 5;
let b = '5';
let x = 7;
let y = 4;
let m = 10;
let n = 9;

console.log("== Equal to (compares only values):");
console.log(`5 == '5' →`, a == b); // true

console.log("\n=== Strict equal (compares value + type):");
console.log(`5 === '5' →`, a === b); // false

console.log("\n!= Not equal (value only):");
console.log(`5 != '5' →`, a != b); // false

console.log("\n!== Strict not equal (value + type):");
console.log(`5 !== '5' →`, a !== b); // true

console.log("\n> Greater than:");
console.log(`7 > 4 →`, x > y); // true

console.log("\n< Less than:");
console.log(`7 < 4 →`, x < y); // false

console.log("\n>= Greater or equal:");
console.log(`10 >= 10 →`, m >= 10); // true

console.log("\n<= Less or equal:");
console.log(`5 <= 9 →`, a <= n); // true
```

String Operators

These are mainly used to join strings together.

Example :

Operator	Description	Example	Output
+	Concatenates strings	"Hello" + " World"	"Hello World"
+=	Appends and assigns	msg += "!"	"Hello!"

Example :

```
let greeting = "Hello";
let name = "John";
let message = greeting + " " + name; // Combines both strings
console.log("Using + :", message);   // Output: Hello Aakash

// Using += operator (Append and assign)
let msg = "Welcome";
msg += " to JavaScript!";  // Appends new text to existing string
console.log("Using += :", msg); // Output: Welcome to JavaScript!
```

Logical Operators

Logical operators combine multiple conditions and return a Boolean result.

Operator	Meaning	Returns True When	Example
&&	Logical AND	All conditions are true	x > 5 && y < 10

||	Logical OR	At least one condition is true	x > 5 || y < 10
!	Logical NOT	Condition is false	!isActive

Example :

```
// Logical AND (&&)
let x = 8;
let y = 6;

if (x > 5 && y < 10) {
  console.log("Both conditions are true (AND).");
} else {
  console.log("One or both conditions are false.");
}

// Logical OR (||)
let a = 4;
let b = 12;

if (a > 5 || b < 15) {
  console.log("At least one condition is true (OR).");
} else {
  console.log("Both conditions are false.");
}

// Logical NOT (!)
let isActive = false;

if (!isActive) {
  console.log("Condition is false, so NOT makes it true.");
} else {
  console.log("Condition was true originally.");
}
```

Conditional (Ternary) Operator

The only operator that takes three operands. It is a shorthand for an `if...else` statement.

Syntax :

condition ? expressionIfTrue : expressionIfFalse;

If the `condition` is `true`, the first expression is evaluated and returned; otherwise, the second expression is evaluated and returned.
Example :

```
let number = 24;
let result = (number % 2 === 0) ? "Even" : "Odd";
console.log(result); // Even
```

Type Operators

Used to determine the data type of a variable or to create a new object.

Operator	Purpose	Example	Output
typeof	Returns variable's data type	typeof "Hi"	"string"
instanceof	Checks if object belongs to a class	arr instanceof Array	true

Example :

```
// typeof examples
let my_name = "John";
let age = 25;
let isStudent = true;

console.log(typeof my_name);      // "string"
console.log(typeof age);       // "number"
console.log(typeof isStudent); // "boolean"

// instanceof examples
let arr = [1, 2, 3];
let today = new Date();

console.log(arr instanceof Array);   // true
console.log(arr instanceof Object);  // true
console.log(today instanceof Date);  // true
console.log(today instanceof Object); // true
```

Bitwise Operators

Operate directly on **binary bits** of numbers.
Useful in advanced programming, graphics, and encryption.

Operator	Description	Example	Output
&	AND	5 & 1	1
|	OR	5 | 1	5

^	XOR	5 ^ 1	4
~	NOT	~5	-6
<<	Left shift	5 << 1	10
>>	Right shift	5 >> 1	2

Example :

```
// Bitwise AND (&)
let a = 6;   // 110 in binary
let b = 3;   // 011 in binary
console.log("6 & 3 =", a & b); // 2 (110 & 011 = 010)

// Bitwise OR (|)
console.log("6 | 3 =", a | b); // 7 (110 | 011 = 111)

// Bitwise XOR (^)
console.log("6 ^ 3 =", a ^ b); // 5 (110 ^ 011 = 101)

// Bitwise NOT (~)
console.log("~6 =", ~a); // -7 (invert all bits)

// Left shift (<<)
console.log("6 << 1 =", a << 1); // 12 (110 shifted left = 1100)

// Right shift (>>)
console.log("6 >> 1 =", a >> 1); // 3 (110 shifted right = 011)
```

Unary Operators

A unary operator is an operator that works on only one operand to perform an operation, such as incrementing/decrementing a value, negating a boolean, or checking the type of a variable.

Operator	Action	Example	Output
typeof	Shows data type	typeof 10	"number"
++	Increments value	++x	Increases by 1
--	Decrements value	--x	Decreases by 1
!	Logical NOT	!true	false

Nullish Coalescing Operator (??)

Nullish Coalescing Operator introduced in ES2020 that returns the right-hand value only if the left-hand value is `null` or `undefined`.

Example :

```
let username = null;
let defaultName = "Guest";

// If username is null or undefined, defaultName will be used
```

```
let displayName = username ?? defaultName;

console.log(displayName);
```

Math in JS:

`Math` object is a built-in object that provides various mathematical constants, functions, and methods to perform complex calculations easily.
It helps you perform operations like rounding numbers, generating random values, finding square roots, trigonometric calculations, and much more — without needing to import any library.

1. The `Math` object is not a constructor, meaning you can't create instances like `new Math()`.
2. You access its properties and methods directly using `Math.`
   Example:
   `Math.sqrt(25); // 5`
3. It always works with numbers and returns numeric values.

Math Properties / Constant :

Math constants are predefined fixed numeric values that represent important mathematical quantities such as π (pi), e (Euler's number), and logarithmic or square root values.
They help you perform precise mathematical calculations without manually typing long decimal values

Property	Description	Example
Math.PI	Returns the value of π (pi) ≈ 3.14159	Math.PI

Math.E	Euler’s number ≈ 2.718	Math.E
Math.SQRT2	Square root of 2	Math.SQRT2
Math.SQRT1_2	Square root of ½	Math.SQRT1_2
Math.LN2	Natural logarithm of 2	Math.LN2
Math.LN10	Natural logarithm of 10	Math.LN10
Math.LOG2E	Base 2 logarithm of E	Math.LOG2E
Math.LOG10E	Base 10 logarithm of E	Math.LOG10E

Example :

```
console.log("Math.PI:", Math.PI);          // Value of π ≈ 3.14159
console.log("Math.E:", Math.E);            // Euler's number ≈ 2.718
console.log("Math.SQRT2:", Math.SQRT2);    // Square root of 2
console.log("Math.SQRT1_2:", Math.SQRT1_2);// Square root of 1/2
console.log("Math.LN2:", Math.LN2);        // Natural logarithm of 2
console.log("Math.LN10:", Math.LN10);      // Natural logarithm of 10
console.log("Math.LOG2E:", Math.LOG2E);    // Base 2 logarithm of E
console.log("Math.LOG10E:", Math.LOG10E);  // Base 10 logarithm of E

let squareSide = 8;
let squareDiagonal = Math.SQRT2 * squareSide; // Diagonal of a square
console.log("Diagonal of square with side 8:", squareDiagonal.toFixed(2));
```

```
let circleCircumference = 2 * Math.PI * 7; // Circumference of circle with radius 7
console.log("Circumference of circle with radius 7:", circleCircumference.toFixed(2));

let naturalExp = Math.E ** 3; // e cubed
console.log("e cubed:", naturalExp.toFixed(3));

let logExample = Math.log(32) / Math.LN2; // log base 2 of 32
console.log("log2(32):", logExample);
```

Math Methods :

Math methods are built-in functions inside the Math object that perform specific mathematical operations — such as rounding numbers, finding powers, square roots, absolute values, or generating random numbers.

Method	Description	Example	Output
Math.round(x)	Rounds to the nearest integer	Math.round(4.6)	5
Math.floor(x)	Rounds down	Math.floor(4.9)	4
Math.ceil(x)	Rounds up	Math.ceil(4.1)	5
Math.trunc(x)	Removes the decimal part	Math.trunc(7.8)	7
Math.pow(x, y)	Raises x to the power y	Math.pow(2, 3)	8

Math.sqrt(x)	Returns the square root	Math.sqrt(81)	9
Math.abs(x)	Returns the absolute value	Math.abs(-12)	12
Math.min(a, b, ...)	Returns the smallest value	Math.min(5, 3, 9)	3
Math.max(a, b, ...)	Returns the largest value	Math.max(5, 3, 9)	9
Math.random()	Returns a random number between 0 (inclusive) and 1 (exclusive)	Math.random()	Example: 0.547
Math.sign(x)	Returns 1 if positive, -1 if negative, 0 if zero	Math.sign(-25)	-1

Example:

```
// Math.round() - rounds to nearest integer
console.log("Math.round(6.4):", Math.round(6.4)); // 6

// Math.floor() - rounds down
console.log("Math.floor(7.8):", Math.floor(7.8)); // 7

// Math.ceil() - rounds up
console.log("Math.ceil(4.2):", Math.ceil(4.2)); // 5

// Math.trunc() - removes decimal part
console.log("Math.trunc(9.9):", Math.trunc(9.9)); // 9
```

```
// Math.pow() - raises to power
console.log("Math.pow(3, 2):", Math.pow(3, 2)); // 9

// Math.sqrt() - square root
console.log("Math.sqrt(64):", Math.sqrt(64)); // 8

// Math.abs() - absolute value
console.log("Math.abs(-15):", Math.abs(-15)); // 15

// Math.min() - smallest value
console.log("Math.min(12, 7, 20):", Math.min(12, 7, 20)); // 7

// Math.max() - largest value
console.log("Math.max(12, 7, 20):", Math.max(12, 7, 20)); // 20

// Math.random() - random number between 0 and 1
let randomNum = Math.random();
console.log("Math.random():", randomNum.toFixed(3)); // e.g., 0.547

// Math.sign() - sign of number
console.log("Math.sign(-30):", Math.sign(-30)); // -1
console.log("Math.sign(0):", Math.sign(0));    // 0
console.log("Math.sign(25):", Math.sign(25));  // 1
```

Example 2: Random number generator
// Generate a random number between 1 and 100

```
let randomNum = Math.floor(Math.random() * 100) + 1;
console.log("Random number between 1 and 100:", randomNum);
```

Explanation:

- `Math.random()` → gives a decimal between 0 and 1 (e.g., 0.5487)
- Multiply by 100 → gives 0–99.99 range
- `Math.floor()` → removes the decimal part

- Add +1 → shifts the range from 1–100 instead of 0–99

Example 2 : Random RGB color generator

```
function getRandomColor() {
  let r = Math.floor(Math.random() * 256);
  let g = Math.floor(Math.random() * 256);
  let b = Math.floor(Math.random() * 256);
  let color = `rgb(${r}, ${g}, ${b})`;
  console.log("Random Color:", color);
}

getRandomColor();
```

Control Structures

Control structures in JavaScript are statements that control the flow of program execution — meaning they decide which part of the code should run, how many times, or under what condition.

They allow you to make decisions, repeat actions, or branch your logic, depending on conditions or data.

Types of Control Structures

JavaScript has three main categories of control structures:

1. Conditional (Decision-Making) Statements
2. Looping (Iteration) Statements
3. Jump (Branching) Statements

Conditional Statements

These statements execute different code blocks based on conditions.

1. if Statement

Executes a block of code only if the given condition is true.

```
let age = 18;
if (age >= 18) {
  console.log("You are eligible to vote");
}
```

2. if...else Statement

Executes one block if the condition is true, otherwise another.

```
let age = 16;
if (age >= 18) {
  console.log("You can vote");
} else {
  console.log("You are not eligible to vote");
}
```

3. if...else if...else Statement

Used to check multiple conditions in sequence.

```
let marks = 75;

if (marks >= 90) {
  console.log("Grade: A");
```

```
} else if (marks >= 70) {
  console.log("Grade: B");
} else {
  console.log("Grade: C");
}
```

4. switch Statement

Simplifies multiple condition checking based on a single expression.

```
let day = 3;
switch (day) {
  case 1: console.log("Monday"); break;
  case 2: console.log("Tuesday"); break;
  case 3: console.log("Wednesday"); break;
  default: console.log("Invalid day");
}
```

Note:
`break` stops the execution after a matching case;
`default` runs if no case matches.

Looping (Iteration) Statements

These statements repeat a block of code multiple times, as long as a condition remains true.

1. for Loop

Used when you know how many times to repeat.

```
```

```
for (let i = 1; i <= 5; i++) {
  console.log("Count:", i);
}
```

2. while Loop

Executes code as long as a condition is true.

```
let i = 1;
while (i <= 5) {
  console.log(i);
  i++;
}
```

3. do...while Loop

Runs the loop at least once, even if the condition is false.

```
let num = 1;
do {
  console.log(num);
  num++;
} while (num <= 3);
```

4. for...in Loop

Iterates through object properties.

```
```

```
let student = { name: "John", age: 20, grade: "A" };

for (let key in student) {
  console.log(key + ": " + student[key]);
}
```

5. for...of Loop

Iterates through iterable values (like arrays or strings).

```
let fruits = ["apple", "banana", "cherry"];

for (let fruit of fruits) {
  console.log(fruit);
}
```

Jump Statements

These control how or when to exit a loop or function.

Statement	Purpose	Example
break	Exits a loop immediately.	if(i == 5) break;
continue	Skips the current iteration and continues with the next.	if(i == 3) continue;

return	Exits a function and returns a value.	return x + y;

Example :

```
function processNumbers() {
    let numbers = [1, 2, 3, 4, 5, 6];

    console.log("Loop with continue:");
    for (let i = 0; i < numbers.length; i++) {
        if (numbers[i] === 3) {
            continue; // Skip number 3
        }
        console.log(numbers[i]);
    }

    console.log("\nLoop with break:");
    for (let i = 0; i < numbers.length; i++) {
        if (numbers[i] === 5) {
            break; // Stop loop when number is 5
        }
        console.log(numbers[i]);
    }

    console.log("\nFunction with return:");
    let sum = addTwoNumbers(10, 20);
    console.log("Sum:", sum);

    function addTwoNumbers(a, b) {
        return a + b; // Return the sum
    }
}
processNumbers();
```